Where Sensors Are Blind:

Finding Water Leaks with Citizen Reports and a Careful Investigation Agent

Team AgentiX | LMA Group Project 2026 | Project Proposal

Hardik Kothari (2025201046) | Gaurav Patel (2025201065)
Harshal Karangale (2025201060) | Rohit Rohon (2025202035)

1. Domain and Target User

Domain: Urbanization and smart city infrastructure, focusing on urban water supply and water utilities.

Target user: the control-room operator at a water utility who decides where to send a repair crew to dig. Field crews benefit too, because they get fewer wasted trips.

2. Problem Statement

Water leaks are a growing problem in fast-growing cities, where old underground pipes carry more water every year. Most utilities can watch their network at only a few points: a typical city zone of 500 to 3,000 homes [6] is usually monitored by a flow meter at its inlet and only a handful of pressure sensors, because sensors, installation and data links are costly. With so few sensors, a small pressure drop can be explained by many different places in the town, so the control room cannot tell where to dig. Crews then search by walking and listening along pipes, which is slow and often ends with digging in the wrong spot. Meanwhile, citizens already report leaks ("water coming up near the school"), and most new leaks are first found this way [5], but these reports are vague, often point to where water collects downhill, and are sometimes wrong. Today these two sources are handled separately, and general AI tools cannot combine them safely (Section 2.1). We propose an AI investigation system that joins the few available sensors with citizen reports, checks every possible leak location with a water network simulator, collects extra evidence when unsure, and sends a crew only after evidence and human-approval checks. The goal is to find leaks faster and cut wasted digs in the few-sensor setting that most cities actually have.

2.1 Why existing LLM, RAG and agent approaches do not solve this

- **A plain LLM cannot do the physics.** Finding a leak means working out how a pressure change spreads through hundreds of connected pipes. An LLM can only guess. Recent work on AI leak diagnosis in L-Town found the right zone in only 5 of 33 cases when forced to answer, and its safer version refused to answer in 88% of cases [8].
- **RAG alone has nothing to retrieve.** The leak location is not written in any document. It must be worked out from live sensor readings and fresh reports.
- **Current water agents only follow orders.** LLM-EPANET [10] and EPANET-Agentic [9] run simulations when an engineer asks. They do not plan an investigation, do not use citizen reports, and have no checks before a real-world action.
- **General tool-using agents act too quickly.** In ToolEmu, even the safest agent tested made risky mistakes in 23.9% of cases [11]. A dig costs a crew day and closes a road, so acting on weak evidence is not acceptable.

What is needed is a system that computes with physics, understands human reports, investigates step by step and checks itself before acting. This is what we build.

3. Research Questions

> **RQ1.** Under sparse sensing constraints, does integrating citizen reports with simulation-guided hypotheses improve localization accuracy and cut false digs relative to traditional sensor-only baselines?

Hypothesis 1: our system will give a smaller distance error and a lower false-dig rate than sensor-only methods at every sensor count, with the biggest gain when sensors are fewest. **Test:** the same 3, 5 and 10 sensors on the L-Town network for every method.

> **RQ2.** Does our investigation agent identify the correct leak location more often and make fewer unsafe or unapproved actions than a standard tool-using LLM agent, when both agents use the same tools and have the same step budget?

Hypothesis 2: our agent will reach more correct digs and make fewer unsafe or unapproved actions than a standard tool-using agent. **Test:** both agents get the same model, tools and step budget, on the same cases, including trap cases such as fake reports and text that tries to skip approval.

4. Why This Problem Matters

- **Huge water loss.** About **126 billion m³** of treated water is lost worldwide every year, worth about **USD 39 billion** [4]. Leaks are a large part of this.
- **Cities are growing fast.** About 68% of the world's people are expected to live in cities by 2050 [12]. More people means more demand on the same ageing pipes, so leaks will become more common and more costly.
- **Wrong digs are expensive.** Each wrong dig wastes a crew day, blocks traffic and needs the road fixed again, while the real leak keeps running.
- **A free signal is being wasted.** Most new leaks and bursts are first noticed because someone reports them [5], yet leak-finding software hardly uses these reports.
- **It works with what cities already have.** Our system needs no new sensor network. It uses the few sensors a city already owns plus reports it already receives, so it can help many cities, not only rich ones.

5. What Is Missing in Existing Approaches

- **Sensor-only methods are built for dense sensor networks.** Methods such as LILA [2] were designed for networks with many sensors. With only a few sensors, many places give almost the same pressure change, and these methods ignore citizen reports completely.
- **Report hot-spot maps ignore how water flows.** Sending a crew to the centre of a cluster of reports is simple, but it is fooled by water running downhill and by false reports.
- **Past work that mixed reports with sensors still needed many sensors.** Han et al. [7] combined tweets with 218 sensors, did not collect any new evidence and used no language model.
- **No method decides what to check next or checks itself before dispatch.** Existing tools give one answer and stop. None of them asks "what is the cheapest extra evidence that would make us sure?" before a crew is sent.

6. The Five Capabilities: What We Use and Why

We use all five, but each one has a clear job. Where a simpler method is enough, we say so and test it as a baseline.

Capability	Job in our system	Honest reason we need it	How we measure it
Fine-tuning	A small open model (about 3B, QLoRA) reads messy report text and fills in place, what was seen and time.	Reports arrive all the time and contain personal details, so the reader must be fast and run locally inside the utility. A general model does not know local landmarks or how people describe leaks; fine-tuning teaches it this.	Field accuracy (F1) for place, symptom and time
RAG	The agent looks up utility rules (dig rules, who must approve) and past repair history, such as streets with repeated false alarms.	These are documents that differ between cities and change over time. Pipe facts (length, height) are not RAG; they come from a database tool.	Rule violations with vs without RAG
Reasoning	The agent keeps a list of possible leak spots, weighs mixed evidence and decides whether more evidence is worth it.	The scoring maths is done in plain code, not by the LLM. The LLM is used where judgement over text and choices is needed, and to explain the decision to the operator.	Correct-dig rate; steps to decision; step-by-step vs one-shot

Capability	Job in our system	Honest reason we need it	How we measure it
Agentic behaviour	A loop: look at the ranked list, choose the next action, update, stop when the gate passes.	A one-shot answer misses cheap evidence (one temporary sensor, one follow-up question) that can be collected before an expensive dig.	False-dig rate with vs without the loop
Tool calling	Tools: run simulation, get ranked list, place temporary sensor, ask the reporter, query pipe database, search documents, request dig.	Physics must come from a real simulator, not from the LLM guessing. The dig tool only works after approval.	Tool-call accuracy; unsafe or unapproved call rate

What the LLM does not do: the leak search itself is a simulator plus simple probability, written in normal code. A fixed-rule pipeline (no LLM) is one of our baselines, so we can show what the LLM parts really add.

7. Proposed Idea and Architecture

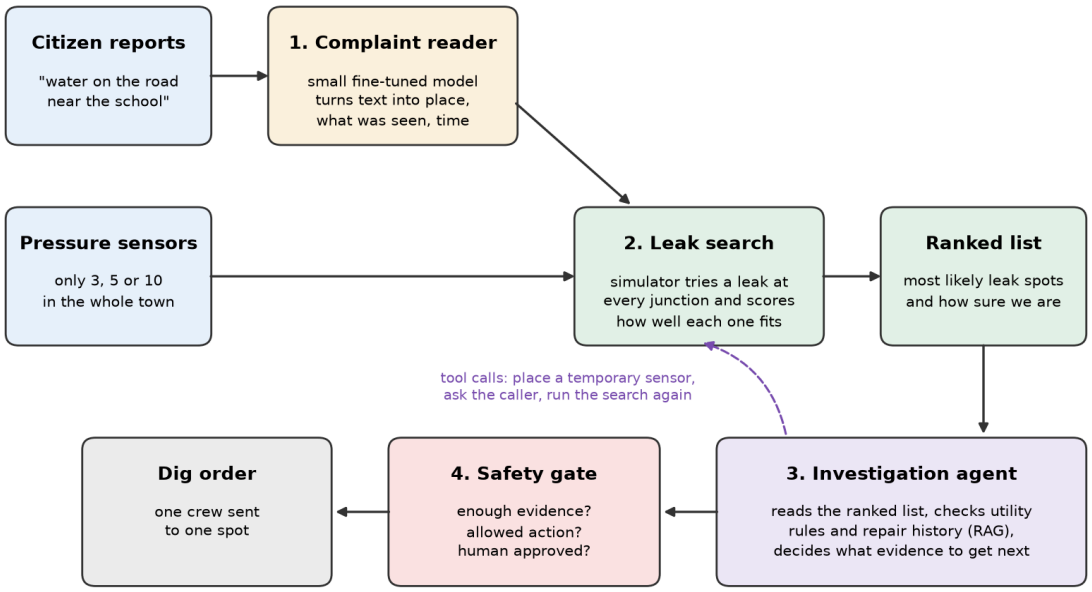


Figure 1. How a leak case moves through the system.

1. **Complaint reader.** Turns each report into a place (street or landmark), what was seen (water on the road, low pressure, no water) and time.
2. **Leak search.** Before any event, we simulate a small leak at each of the 782 junctions once and save the pressure pattern it causes. For a new event, every junction gets a score for how well its pattern matches the sensors, plus a score from the reports. The report score allows for water running downhill and for false reports, so one report can never decide the answer alone.
3. **Investigation agent.** Reads the ranked list. If the top spots are close in score, it chooses the cheapest useful next step, for example placing one temporary pressure sensor where the top spots disagree most, or asking a reporter a follow-up question. It then runs the search again.
4. **Safety gate.** Written in code, not left to the LLM. A dig order goes out only if the top spot is likely enough, the action is on the allowed list, and a human operator approves. Otherwise the system says "not sure yet" and shows what it would check next.

8. Expected Novelty

- **Citizens as sensors, checked by physics.** To our knowledge, this is the first system that treats citizen reports as real evidence and checks each one against a hydraulic simulator, instead of trusting or ignoring them.
- **Built for the real-world sensor count.** Real towns have only a handful of pressure sensors, so we design and test for 3 to 10 sensors, the setting where utilities need help most.
- **An agent that investigates like an engineer.** It keeps a list of possible leak spots and collects the cheapest useful evidence before acting, instead of giving a one-shot answer.

- **Safe by design.** Every dig must pass an evidence check and human approval, and we test this with trap cases. Current LLM water tools have no such safeguard.
- **All five capabilities on a physical system where actions cost money.** Fine-tuning, RAG, reasoning, agentic behaviour and tool calling work together on a real infrastructure problem, and every recommendation comes with a simulator check the operator can see.

9. Dataset and Evaluation Environment

- **L-Town (BattLeDIM 2020) [1], main network:** 782 junctions, 905 pipes, 42.6 km of pipe, about 10,000 people, 33 pressure sensors, 23 leaks in the 2019 test year. Open licence (CC-BY 4.0). We use only 3, 5 or 10 of its sensors, to match the few-sensor setting of real towns.
- **C-Town, second network** (388 junctions, 429 pipes), to check that results carry over to a different town.
- **Simulator:** WNTR [3] (Python, based on EPANET). One 24-hour run takes under one second on a laptop.
- **Citizen reports:** made from each simulated leak. We control whether water reaches the surface, how many people report, how far the report shifts downhill, location noise, wrong streets and false reports. Text is written from templates and then reworded by an LLM. The team will also write about 200 reports by hand as a separate test set for the reader.
- **Test cases:** about 300 leaks per network and sensor setting, plus about 50 trap cases for the agent. The true leak location is known for every case.

10. Baselines

For	Baseline	Why it is a fair comparison
RQ1	LILA [2]	Open Python code, placed 3rd in the BattLeDIM competition; a real method that runs on a laptop.
RQ1	Sensitivity matching	Classic method: match the pressure change to precomputed leak patterns. Same as our search without reports.
RQ1	Report hot spot	Dig at the centre of the report cluster. Close to what utilities do by hand.
RQ2	Standard tool-using agent (ReAct style [13])	Same model, tools and step budget; no hypothesis list and no gate.
RQ2	Plain LLM, no tools	Gets the same case as text and answers once.
RQ2	Fixed rules, no LLM	Always place one sensor at the most uncertain spot, then dig the top spot. Tests what the LLM adds.

11. Metrics

Metric	What it tells us	Used for
Median distance error (m)	How far the top guess is from the real leak	RQ1
Found within 100 m (%)	How often a crew would dig in the right street	RQ1
False-dig rate (%)	Digs more than 100 m from the leak	RQ1, RQ2
Correct-dig rate (%)	Cases that end with a dig within 100 m	RQ2, reasoning
Unsafe or unapproved actions (%)	Actions outside the allowed list or without approval	RQ2, safety
Tool-call accuracy (%)	Right tool with valid inputs	Tool calling
Field F1	Quality of the complaint reader	Fine-tuning
Rule violations (%)	Whether utility rules were followed	RAG
Steps and time per case	How quickly the agent reaches a decision	Agentic behaviour

Ablations

- **No citizen reports:** sensors only, to measure what reports add (RQ1).
- **No temporary-sensor tool:** the agent cannot collect new pressure data.
- **No RAG:** the agent works without utility rules and repair history.

- **No safety gate:** the agent can send a crew without the evidence and approval checks (RQ2).
- **No step-by-step reasoning:** the agent gives one answer in a single step.

Fair statistics: every method is run on the same leaks with 3 random sensor layouts per setting. We report the mean with 95% bootstrap confidence intervals.

12. Compute Plan

Item	Plan	Budget
Simulations	Leak patterns for every junction, all networks and settings, on a laptop CPU (about 3 minutes per network)	under 8-12 CPU hours
Fine-tuning	QLoRA on a 3B model, one 16 GB GPU (Colab or university T4), about 2 h per run, 3 to 5 runs	about 10 GPU hours
Agent runs	About 300 cases x 3 sensor settings x 3 agent types = 2,700 runs, about 10 LLM calls each, mostly on free-tier LLM APIs	about 27,000 API calls
Report rewording	About 3,000 short report texts, one call each, on a free-tier API	about 3,000 API calls
Total		**10 GPU hours, 8-12 CPU hours, about 30,000 API calls**

13. Key Technical Risks and Mitigations

Risk	What could go wrong	How we handle it
Fake vs real reports	Reports written by an LLM may be cleaner and easier to read than real phone calls or social media posts.	We create reports from how water actually flows, then add location noise, typos and wrong streets. We also test the complaint reader on about 200 reports written by hand.
Silent underground leaks	Some breaks never bring water to the surface, so nobody reports them.	When there are no reports, the system automatically uses the pressure sensors alone and the investigation continues normally.
LLM reasoning vs tool use	Better results may come only from placing temporary sensors, not from the LLM's reasoning.	We compare our agent with a fixed rule-based script (no LLM) that has exactly the same sensor tools. The difference shows what the LLM really adds.

References

[1] Vrachimis, S. et al. (2020). BattLeDIM: Battle of the Leakage Detection and Isolation Methods, L-Town dataset. Zenodo 4017659.

[2] Daniel, I. et al. (2022). A sequential pressure-based algorithm for data-driven leakage identification and model-based localization in water distribution networks (LILA). J. Water Resources Planning and Management.

[3] Klise, K. et al. WNTR: Water Network Tool for Resilience, Python package. Sandia National Laboratories and US EPA.

[4] Liemberger, R. and Wyatt, A. (2019). Quantifying the global non-revenue water problem. Water Supply.

[5] Lambert, A. (2001). Water losses management and techniques. IWA International Report, Berlin.

[6] International Water Association (2024). District Metered Areas: Guidance Notes, Version 2.

[7] Han et al. (2017). Leak event identification in water systems using high order CRF. arXiv 1703.04170.

[8] Mu, T. et al. (2026). Verifiable abstention makes AI leak diagnosis accountable in urban water distribution networks. arXiv 2608.18836.

[9] Wang, J., Fu, G. and Savic, D. (2026). EPANET-Agentic: a multi-agent system for natural language-controlled simulations of water distribution networks. Water Research.

[10] LLM-EPANET: an agent-based framework for natural language interaction with EPANET. arXiv 2503.16191.

[11] Ruan, Y. et al. (2024). Identifying the risks of LM agents with an LM-emulated sandbox (ToolEmu). ICLR.

[12] United Nations DESA (2018). World Urbanization Prospects: The 2018 Revision.

[13] Yao, S. et al. (2023). ReAct: Synergizing reasoning and acting in language models. ICLR.